Sentinel: local vulnerability monitoring validation

Libor Burian | Engineering report | 22 October 2026

Abstract

Sentinel is a local Windows and WSL vulnerability-monitoring dashboard. It combines collected installation metadata, CISA known-exploitation intelligence, OSV project dependency results, and web research through authenticated Codex or Claude Code processes. This report evaluates integration behavior and evidence handling on one device. It does not estimate detection accuracy. Automated checks passed, both provider conversation sessions resumed, and live research produced structured installation assessments. One malformed inventory reference and one incorrect exploitation category in the Luna output required explicit handling; these errors are part of the results.

Context, motivation, and background

Vulnerability news is insufficient to establish exposure of a particular installation. Windows cumulative updates, distribution backports, separate WSL environments, and lockfile-versus-runtime distinctions complicate applicability. The dashboard makes those distinctions visible and proposes copyable fixes with execution context. It does not execute remediation or assert whole-device safety. Inventory choices follow Microsoft's guidance, and dependency parsing is delegated to OSV-Scanner rather than implemented anew [1, 2].

Method and architecture

The sequence is inventory collection, intelligence/dependency collection, agent investigation, structural evidence validation, and persistence/publication. Python, uv, FastAPI, Pydantic, SQLite/WAL, and native browser modules provide the application. A shared operating-system file lock prevents simultaneous scans across manual, app-timer, and standalone entry points. Persisted server-sent events support reconnection [3].

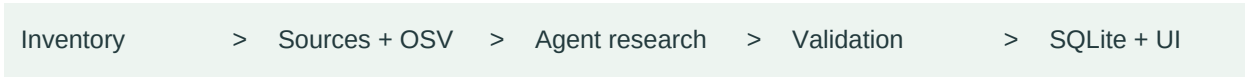

Figure 1. Evidence flows through five phases; raw agent output is logged before validated findings are published.

Windows collection reads OS/build, uninstall registry, current-user AppX, and supplemental KB evidence. WSL collection separates platform/kernel from distribution release and dpkg binary/source packages. Only explicitly configured project roots are scanned. CISA KEV is fetched with its official mirror as fallback; agents additionally perform live research. Codex uses JSONL, an explicit model and final schema, live web search, and bypassed approval/sandbox settings. Claude uses streamed print mode, an explicit model and schema, permission bypass, and disabled sandbox while retaining subscription authentication [4, 5].

Pre-implementation acceptance criteria concerned reproducible behavior: reject unsupported definitive assessments and unknown inventory references; preserve prior findings on failures; retain complete emitted logs; resume explicit sessions; and avoid duplicate scan owners. Tests use synthetic fixtures for those boundaries. The live checks are a single-machine integration case study, with retrospective source spot checks. There is no training, held-out detection benchmark, random sampling, seed study, or statistical comparison. No inference about precision, recall, or general model superiority is warranted.

Results: software checks and Luna

The observed final local software check was **88 passing tests** in 8.46 seconds. All six frontend modules parsed and linked. The Windows CI run for the earlier commit e92a2c1 also completed successfully. Tests cover backport-sensitive prompts/evidence, incomplete Windows update data, separate installations, scanner exit codes, unavailable sources, alias consolidation, invalid results, long events, concurrent stderr, timeouts, process-tree cancellation, restart recovery, scan-lock collisions, and primary WSL range contradictions with matching news caveats. These are behavioral assertions, not a count of real vulnerabilities detected.

The Luna run requested `gpt-6-luna`, completed its original research in 965.773 seconds, and emitted 23 distinct web-tool IDs. That count includes web operations and does not count independent queries. The final JSON contained 12 advisories and 15 proposed findings. Exact asset binding rejected one identifier with a missing character, retaining 14 findings: four affected, four potentially affected, and six not affected. Replaying the unchanged output through the corrected publication gate changed the recorded workflow status to partial; it did not perform new research.

Two retrospective applicability checks were supported by their official sources: a distribution-specific Ubuntu c-ares patch threshold and an ImageMagick Windows affected-release range [6, 7]. A separate 7-Zip source check found a public proof-of-concept generator in the cited advisory although Luna had assigned none-known exploitation. The stored category was corrected to public proof of concept with a timestamped audit event, and prompt guidance was clarified [8]. The original model output was retained. These examples demonstrate that schema compliance is not factual verification.

Results: provider sessions and Haiku workflow

Haiku's initial inventory conversation and explicit resumed turn completed using the CLI-reported `claude-haiku-5-5` model and the same provider session identifier. Luna's two conversation turns also retained the same explicit session. Answers about inventory counts and update/backport limitations agreed with the supplied snapshot. Complete ordered event text in the compared JSONL files matched SQLite, and final assistant text matched the saved messages. This establishes those observed session/logging behaviors, not long-context reliability.

The full Haiku dashboard run completed its original workflow in 266.940 seconds, with nine WebSearch and three WebFetch tool IDs, eight advisories, and ten schema-valid findings. It inspected a fresh Windows/WSL inventory and 47 recorded project dependencies. Primary-source review contradicted two definitive WSL findings: Microsoft's CNA records contain inverted lower/upper bounds while a secondary derived range appeared usable [10]. Both were qualified as potentially affected and the workflow was marked partial. The ten final reviewed findings comprise zero affected, six potential, three not affected, and one unknown. Additional review annotations corrected an inventory-absence overclaim and a browser stable-channel statement. A narrow primary-range check was then added; replaying the unchanged original output with live CNA sources automatically reproduced both downgrades without another model call. This checks source consistency, not installed-version applicability or general detection accuracy.

Interface and operational observations

Desktop and 390-pixel mobile layouts were inspected. Combined installation/source/severity filters, empty filtered results, finding evidence, remediation metadata, and finding-scoped chat were exercised. The copy action reported browser API success without command execution; the automation bridge did not independently confirm system clipboard contents. Reload and restart retained persisted state. A disabled temporary Windows scheduling task was registered, inspected, and removed; login/reboot execution was not tested. The public README screenshot uses a real empty workspace and contains no device evidence.

Discussion, limitations, and implications

The implementation makes uncertainty and failure inspectable, but coverage remains selected and incomplete. Portable software, other users' profiles, non-dpkg distributions, arbitrary language environments, runtime reachability, and every possible advisory are not exhaustively inspected. Live web results and provider access change over time. An observed web-tool event establishes tool use, not research completeness or correctness. The agents run with the requested unrestricted permissions; prompts describe the task but do not enforce filesystem or network restrictions.

No exploits or remediation commands were intentionally executed during validation. Transcript review is not a complete filesystem audit. The ordinary frontend has no remediation execution endpoint. Full local logs may include paths and provider diagnostics; they are ignored by Git and are not redacted or rotated by the app. A completed scan denotes workflow completion without recognized structural gaps, not proof that the device is safe.

The source spot checks were chosen after viewing the output and are not statistically representative. Their conclusions are limited to the cited version/evidence relationships. No confidence interval or significance test is meaningful for detection accuracy without a defined labeled population and sampling plan. Model-internal architecture and training were not examined.

Conclusion and reproducibility

The evidence supports a practical local integration: inventory and dependency collection, real agent research, structured findings, durable history, proposed fixes, and session-bound follow-up. It also directly shows why users must inspect citations and uncertainty. Clone the public repository, follow its setup guide, and use the locked uv environment. Automated checks require no model credentials; live scans require the selected authenticated CLI and network access. The detailed sanitized records identify observed runs, corrections, and remaining gaps. The PDF is generated from this Markdown with the repository's ReportLab script.

References and related implementations

- [1] Microsoft: Working with software installations and Get-HotFix limitations.
- [2] Google: OSV-Scanner supported artifacts and output contract. CISA: official KEV data.
- [3] FastAPI: Server-Sent Events.
- [4] OpenAI: Codex non-interactive mode.
- [5] Anthropic: Claude Code programmatic usage.
- [6] Canonical: USN-8951-1.
- [7] ImageMagick: GHSA-x8g2-7r3w-h44p.
- [8] GitHub Security Lab: GHSL-2026-140.
- [9] Repository, source code, tests, and detailed validation records.
- [10] Microsoft CNA records in the official CVE Program repository: CVE-2026-57968 and CVE-2026-57973.